

magician package

Package to control the Dobot Magician using the python programming language.



Submodules

magician.bot module

```
class magician.bot.Alarm(value, names=<not given>, *values, module=None,
qualname=None, type=None, start=1, boundary=None)
```

[\[source\]](#)

Bases: `IntEnum`

Point number codes that represent the alarms used by Dobot Magician.

Parameters:

COMMON_RESETTING : *int*, *default* = 0x00

The alarm will be triggered after resetting system.

COMMON_UNDEFINED_INSTRUCTION : *int*, *default* = 0x01

Receive undefined instruction.

COMMON_FILE_SYSTEM : *int, default = 0x02*

File system error.

COMMON_MCU_FPGA_COMM : *int, default = 0x03*

There is a failed communication between MCU and FPGA when system is initializing

COMMON_ANGLE_SENSOR : *int, default = 0x04*

Get an error value of angle sensor

PLAN_INV_SINGULARITY : *int, default = 0x10*

The target point (beginning point or ending point) is in abnormal position in Cartesian coordinate system.

PLAN_INV_CALC : *int, default = 0x11*

The target point is out of the workplace in all motion mode.

PLAN_INV_LIMIT : *int, default = 0x12*

The inverse resolve of target point is out of the limitation.

PLAN_PUSH_DATA_REPEAT : *int, default = 0x13*

There are some repetitive points in ARC or JUMP_MOVE mode.

PLAN_ARC_INPUT_PARAM : *int, default = 0x14*

Missing manufacturer description.

PLAN_JUMP_PARAM : *int, default = 0x15*

Set a wrong parameter in JUMP mode.

PLAN_LINE_HAND : *int, default = 0x16*

Missing manufacturer description.

PLAN_LINE_OUT_SPACE : *int, default = 0x17*

Missing manufacturer description.

PLAN_ARC_OUT_SPACE : *int, default = 0x18*

Missing manufacturer description.

PLAN_MOTIONTYPE : *int, default = 0x19*

Missing manufacturer description.

PLAN_SPEED_INPUT_PARAM : *int, default = 0x1A*

Missing manufacturer description.

PLAN_CP_CALC : *int, default = 0x1B*

Missing manufacturer description.

MOVE_INV_SINGULARITY : *int*, *default* = 0x20

Motion trajectory is in singularity position that causes inverse resolve alarm in Cartesian coordinate system.

MOVE_INV_CALC : *int*, *default* = 0x21

Motion is out of workspace that causes inverse resolve alarm when Dobot Magician moves.

MOVE_INV_LIMIT : *int*, *default* = 0x22

The inverse resolve is out of the limitation when Dobot Magician moves.

OVERSPEED_AXIS1 : *int*, *default* = 0x30

Joint 1 is overspeed.

OVERSPEED_AXIS2 : *int*, *default* = 0x31

Joint 2 is overspeed.

OVERSPEED_AXIS3 : *int*, *default* = 0x32

Joint 3 is overspeed.

OVERSPEED_AXIS4 : *int*, *default* = 0x33

Joint 4 is overspeed.

LIMIT_AXIS1_POS : *int*, *default* = 0x40

Joint 1 moves to the positive limitation area.

LIMIT_AXIS1_NEG : *int*, *default* = 0x41

Joint 1 moves to the negative limitation area.

LIMIT_AXIS2_POS : *int*, *default* = 0x42

Joint 2 moves to the positive limitation area.

LIMIT_AXIS2_NEG : *int*, *default* = 0x43

Joint 2 moves to the negative limitation area.

LIMIT_AXIS3_POS : *int*, *default* = 0x44

Joint 3 moves to the positive limitation area.

LIMIT_AXIS3_NEG : *int*, *default* = 0x45

Joint 3 moves to the negative limitation area.

LIMIT_AXIS4_POS : *int*, *default* = 0x46

Joint 4 moves to the positive limitation area.

LIMIT_AXIS4_NEG : *int*, *default* = 0x47

Joint 4 moves to the negative limitation area.

LIMIT_AXIS23_POS : *int, default = 0x48*

Parallelogram is stretched to the limitation area.

LIMIT_AXIS23_NEG : *int, default = 0x49*

Parallelogram is stretched to the limitation area.

LOSE_STEP_AXIS1 : *int, default = 0x50*

Joint 1 loses step.

LOSE_STEP_AXIS2 : *int, default = 0x51*

Joint 2 loses step.

LOSE_STEP_AXIS3 : *int, default = 0x52*

Joint 3 loses step.

LOSE_STEP_AXIS4 : *int, default = 0x53*

Joint 4 loses step.

OTHER_AXIS1_DRV_ALARM : *int, default = 0x60*

Missing manufacturer description.

OTHER_AXIS1_OVERFLOW : *int, default = 0x61*

Missing manufacturer description.

OTHER_AXIS1_FOLLOW : *int, default = 0x62*

Missing manufacturer description.

OTHER_AXIS2_DRV_ALARM : *int, default = 0x63*

Missing manufacturer description.

OTHER_AXIS2_OVERFLOW : *int, default = 0x64*

Missing manufacturer description.

OTHER_AXIS2_FOLLOW : *int, default = 0x65*

Missing manufacturer description.

OTHER_AXIS3_DRV_ALARM : *int, default = 0x66*

Missing manufacturer description.

OTHER_AXIS3_OVERFLOW : *int, default = 0x67*

Missing manufacturer description.

OTHER_AXIS3_FOLLOW : *int, default = 0x68*

Missing manufacturer description.

OTHER_AXIS4_DRV_ALARM : *int, default = 0x69*

Missing manufacturer description.

OTHER_AXIS4_OVERFLOW : *int*, *default* = 0x6A

Missing manufacturer description.

OTHER_AXIS4_FOLLOW : *int*, *default* = 0x6B

Missing manufacturer description.

MOTOR_REAR_ENCODER : *int*, *default* = 0x70

Missing manufacturer description.

MOTOR_REAR_TEMPERATURE_HIGH : *int*, *default* = 0x71

Missing manufacturer description.

MOTOR_REAR_TEMPERATURE_LOW : *int*, *default* = 0x72

Missing manufacturer description.

MOTOR_REAR_LOCK_CURRENT : *int*, *default* = 0x73

Missing manufacturer description.

MOTOR_REAR_BUSV_HIGH : *int*, *default* = 0x74

Missing manufacturer description.

MOTOR_REAR_BUSV_LOW : *int*, *default* = 0x75

Missing manufacturer description.

MOTOR_REAR_OVERHEAT : *int*, *default* = 0x76

Missing manufacturer description.

MOTOR_REAR_RUNAWAY : *int*, *default* = 0x77

Missing manufacturer description.

MOTOR_REAR_BATTERY_LOW : *int*, *default* = 0x78

Missing manufacturer description.

MOTOR_REAR_PHASE_SHORT : *int*, *default* = 0x79

Missing manufacturer description.

MOTOR_REAR_PHASE_WRONG : *int*, *default* = 0x7A

Missing manufacturer description.

MOTOR_REAR_LOST_SPEED : *int*, *default* = 0x7B

Missing manufacturer description.

MOTOR_REAR_NOT_STANDARDIZE : *int*, *default* = 0x7C

Missing manufacturer description.

ENCODER_REAR_NOT_STANDARDIZE : *int*, *default* = 0x7D

Missing manufacturer description.

MOTOR_REAR_CAN_BROKE : *int, default = 0x7E*

Missing manufacturer description.

MOTOR_FRONT_ENCODER : *int, default = 0x80*

Missing manufacturer description.

MOTOR_FRONT_TEMPERATURE_HIGH : *int, default = 0x81*

Missing manufacturer description.

MOTOR_FRONT_TEMPERATURE_LOW : *int, default = 0x82*

Missing manufacturer description.

MOTOR_FRONT_LOCK_CURRENT : *int, default = 0x83*

Missing manufacturer description.

MOTOR_FRONT_BUSV_HIGH : *int, default = 0x84*

Missing manufacturer description.

MOTOR_FRONT_BUSV_LOW : *int, default = 0x85*

Missing manufacturer description.

MOTOR_FRONT_OVERHEAT : *int, default = 0x86*

Missing manufacturer description.

MOTOR_FRONT_RUNAWAY : *int, default = 0x87*

Missing manufacturer description.

MOTOR_FRONT_BATTERY_LOW : *int, default = 0x88*

Missing manufacturer description.

MOTOR_FRONT_PHASE_SHORT : *int, default = 0x89*

Missing manufacturer description.

MOTOR_FRONT_PHASE_WRONG : *int, default = 0x8A*

Missing manufacturer description.

MOTOR_FRONT_LOST_SPEED : *int, default = 0x8B*

Missing manufacturer description.

MOTOR_FRONT_NOT_STANDARDIZE : *int, default = 0x8C*

Missing manufacturer description.

ENCODER_FRONT_NOT_STANDARDIZE : *int, default = 0x8D*

Missing manufacturer description.

MOTOR_FRONT_CAN_BROKE : *int, default = 0x8E*

Missing manufacturer description.

MOTOR_Z_ENCODER : *int, default = 0x90*

Missing manufacturer description.

MOTOR_Z_TEMPERATURE_HIGH : *int, default = 0x91*

Missing manufacturer description.

MOTOR_Z_TEMPERATURE_LOW : *int, default = 0x92*

Missing manufacturer description.

MOTOR_Z_LOCK_CURRENT : *int, default = 0x93*

Missing manufacturer description.

MOTOR_Z_BUSV_HIGH : *int, default = 0x94*

Missing manufacturer description.

MOTOR_Z_BUSV_LOW : *int, default = 0x95*

Missing manufacturer description.

MOTOR_Z_OVERHEAT : *int, default = 0x96*

Missing manufacturer description.

MOTOR_Z_RUNAWAY : *int, default = 0x97*

Missing manufacturer description.

MOTOR_Z_BATTERY_LOW : *int, default = 0x98*

Missing manufacturer description.

MOTOR_Z_PHASE_SHORT : *int, default = 0x99*

Missing manufacturer description.

MOTOR_Z_PHASE_WRONG : *int, default = 0x9A*

Missing manufacturer description.

MOTOR_Z_LOST_SPEED : *int, default = 0x9B*

Missing manufacturer description.

MOTOR_Z_NOT_STANDARDIZE : *int, default = 0x9C*

Missing manufacturer description.

ENCODER_Z_NOT_STANDARDIZE : *int, default = 0x9D*

Missing manufacturer description.

MOTOR_Z_CAN_BROKE : *int, default = 0x9E*

Missing manufacturer description.

MOTOR_R_ENCODER : *int, default = 0xA0*

Missing manufacturer description.

MOTOR_R_TEMPERATURE_HIGH : *int, default = 0xA1*

Missing manufacturer description.

MOTOR_R_TEMPERATURE_LOW : *int, default = 0xA2*

Missing manufacturer description.

MOTOR_R_LOCK_CURRENT : *int, default = 0xA3*

Missing manufacturer description.

MOTOR_R_BUSV_HIGH : *int, default = 0xA4*

Missing manufacturer description.

MOTOR_R_BUSV_LOW : *int, default = 0xA5*

Missing manufacturer description.

MOTOR_R_OVERHEAT : *int, default = 0xA6*

Missing manufacturer description.

MOTOR_R_RUNAWAY : *int, default = 0xA7*

Missing manufacturer description.

MOTOR_R_BATTERY_LOW : *int, default = 0xA8*

Missing manufacturer description.

MOTOR_R_PHASE_SHORT : *int, default = 0xA9*

Missing manufacturer description.

MOTOR_R_PHASE_WRONG : *int, default = 0xAA*

Missing manufacturer description.

MOTOR_R_LOST_SPEED : *int, default = 0xAB*

Missing manufacturer description.

MOTOR_R_NOT_STANDARDIZE : *int, default = 0xAC*

Missing manufacturer description.

ENCODER_R_NOT_STANDARDIZE : *int, default = 0xAD*

Missing manufacturer description.

MOTOR_R_CAN_BROKE : *int, default = 0xAE*

Missing manufacturer description.

MOTOR_ENDIO_IO : *int, default = 0xB0*

Missing manufacturer description.

MOTOR_ENDIO_RS485_WRONG : *int, default = 0xB1*

Missing manufacturer description.

MOTOR_ENDIO_CAN_BROKE : *int*, default = 0xB2

Missing manufacturer description.

Attributes:**denominator**

the denominator of a rational number in lowest terms

imag

the imaginary part of a complex number

numerator

the numerator of a rational number in lowest terms

real

the real part of a complex number

Methods

as_integer_ratio ()	Return a pair of integers, whose ratio is equal to the original int.
bit_count ()	Number of ones in the binary representation of the absolute value of self.
bit_length ()	Number of bits necessary to represent self in binary.
conjugate	Returns self, the complex conjugate of any int.
from_bytes (/, bytes[, byteorder, signed])	Return the integer represented by the given array of bytes.
is_integer ()	Returns True.
to_bytes (/[, length, byteorder, signed])	Return an array of bytes representing an integer.

References

<https://download.dobot.cc/product-manual/dobot-magician/pdf/en/Dobot-Magician-ALARM-Description.pdf>

```
class magician.bot.Bot(port: str | None = None)
```

[\[source\]](#)

Bases: `object`

Creates an instance of Bot connected to the given serial port.

Parameters:

port: str, optional

Name of the serial port to connect

Methods

<code>check_bounds</code> ([joints])	Validate the bounds of the robot desired end position.
<code>clear_alarms</code> ()	Clears all alarms of the Robot.
<code>close</code> ()	Close serial connection.
<code>get_pose</code> ()	Returns the current position position of the robotic arm, both in the cartesian plane and the joints plane.
<code>grip</code> ([enable])	Control claw grip in the end effector.
<code>home</code> ()	Send the robotic arm to the defined home position.
<code>inc_ang</code> (mode[, j1, j2, j3, j4, wait])	Increment robotic arm position by (j1,j2,j3) in the joints plane and increment end effector angle by j4.
<code>inc_xyz</code> (mode[, x, y, z, r, wait])	Increment robotic arm end effector position by (x,y,z) in the Cartesian plane and increment the end effector rotation servo angle by r.
<code>move_to_ang</code> (mode[, j1, j2, j3, j4, wait, joints])	Move robotic arm to the (j1,j2,j3) position in the joints plane and set the end effector rotation servo angle to j4.
<code>move_to_xyz</code> (mode[, x, y, z, r, wait, position])	Move robotic arm to the (x,y,z) position in the Cartesian plane and set the end

	effector angle to r.
<code>set_home</code> (x, y, z[, r])	Defines the dobot robotic arm home position in the cartesian plane.
<code>suck</code> ([enable])	Control suction in the end effector.

conveyor_belt	
conveyor_belt_distance	
engrave	
get_alarms	
get_color	
get_hht_trig_output	
get_ir	
go_arc	
jog_r	
jog_x	
jog_y	
jog_z	
laze	
set_color	
set_hht_trig_output	
set_io	
set_ir	
speed	
wait_for_cmd	

J1_MAX = 120

Joint 1 (Base) - maximum allowed angle in degrees

J1_MIN = -120

Joint 1 (Base) - minimum allowed angle in degrees

J2_MAX = 90

Joint 2 (Rear arm) - maximum allowed angle in degrees

J2_MIN = -5

Joint 2 (Rear arm) - minimum allowed angle in degrees

J3_MAX = 90

Joint 3 (Forearm) - maximum allowed angle in degrees

J3_MIN = -15

Joint 3 (Forearm) - minimum allowed angle in degrees

J4_MAX = 140

Joint 4 (Rotation servo) - maximum allowed angle in degrees

J4_MIN = -140

Joint 4 (Rotation servo) - minimum allowed angle in degrees

check_bounds(joints: [Joints](#) = None)

[\[source\]](#)

Validate the bounds of the robot desired end position.

Parameters:

joints : *bot.Joints*

The joints object to check.

Returns:

Nothing

Raises:

ValueError

If joints are not provided.

BotBoundsException

If any joint is out of bounds.

clear_alarms() → None

[\[source\]](#)

Clears all alarms of the Robot.

close() → None

[\[source\]](#)

Close serial connection.

↪ See also

`serial.Serial.close`

get_pose() → [Pose](#)

[\[source\]](#)

Returns the current position position of the robotic arm, both in the cartesian plane and the joints plane.

↪ See also

`bot.Pose`

grip(enable=False)

[\[source\]](#)

Control claw grip in the end effector.

Parameters:

enable: bool, default = False

Notes

Will only have effect if the claw module is mounted and the pressure module is connected.

`home()`

[\[source\]](#)

Send the robotic arm to the defined home position.

↪ See also

`bot.set_home`

Notes

This is an important step in the initialization and calibration of the robot.

`inc_ang(mode: MOVE\_TYPE, j1=0, j2=0, j3=0, j4=0, wait=True)`

[\[source\]](#)

Increment robotic arm position by (j1,j2,j3) in the joints plane and increment end effector angle by j4.

Parameters:

mode : *MOVE_TYPE*

Type of movement to be performed. Valid options { `MOVE_TYPE.MOVEJ`, `MOVE_TYPE.MOVL` }.

j1 : *float, optional*

J1 angle change in degrees. Defaults to 0.

j2 : *float, optional*

J2 angle change in degrees. Defaults to 0.

j3 : *float, optional*

J3 angle change in degrees. Defaults to 0.

j4 : *float, optional*

J4 angle change in degrees. Defaults to 0.

wait : *bool, optional*

If True, wait for the previous movement to complete before executing this new one. Defaults to True.

Returns:

Robot ptp movement instruction.

Raises:

ValueError

If an invalid movement type is provided.

Examples

```
>>> inc_ang(MODE_TYPE.MOVJ,x=-10,y=0,z=10)
>>> inc_ang(MODE_TYPE.MOVJ,y=-5)
```

inc_xyz(*mode*: [MOVE_TYPE](#), *x*=0, *y*=0, *z*=0, *r*=0, *wait*=True)

[\[source\]](#)

Increment robotic arm end effector position by (x,y,z) in the Cartesian plane and increment the end effector rotation servo angle by r.

Parameters:

mode : *bot.MOVE_TYPE*

Type of movement to be performed. Valid options { MOVE_TYPE.JUMP, MOVE_TYPE.MOVJ }.

x : *float, optional*

End effector x-coordinate change. Defaults to 0.

y : *float, optional*

End effector y-coordinate change. Defaults to 0.

z : *float, optional*

End effector z-coordinate change. Defaults to 0.

r : *float, optional*

End effector rotation servo orientation change. Defaults to 0.

wait : *bool, optional*

If True, wait for the previous movement to complete before executing this new one. Defaults to True.

Returns:

Robot ptp movement instruction.

Raises:

ValueError

If an invalid movement type is provided.

Examples

```
>>> inc_xyz(MODE_TYPE.MOVJ, x=-10, y=0, z=10)
>>> inc_xyz(MODE_TYPE.MOVJ, y=-5)
```

move_to_ang(mode: [MOVE_TYPE](#), j1=None, j2=None, j3=None, j4=None, wait=True, joints: [Joints](#) = None) [\[source\]](#)

Move robotic arm to the (j1,j2,j3) position in the joints plane and set the end effector rotation servo angle to j4.

Parameters:

mode : *bot.MOVE_TYPE*

Type of movement to be performed. Valid options { MOVE_TYPE.JUMP, MOVE_TYPE.MOVJ, MOVE_TYPE.MOVL }.

j1 : *float, optional*

Target J1 angle. Defaults to the current j1 angle if None.

j2 : *float, optional*

Target J2 angle. Defaults to the current j2 angle if None.

j3 : *float, optional*

Target J4 angle. Defaults to the current j3 angle if None.

j4 : *float, optional*

Target J4 rotation servo angle. Defaults to 0.

wait : *bool, optional*

If True, wait for the previous movement to complete before executing this new one. Defaults to True.

joints : *Joints, optional*

A Joints object containing j1, j2, j3, and j4 values. If provided, overrides j1, j2, j3, and j4 parameters.

Returns:

Robot ptp movement instruction.

Raises:

ValueError

If both joints and (j1, j2, j3) args are None, or if an invalid movement type is provided.

Examples

```

>>> move_to_ang(MODE_TYPE.MOVJ,x=-100,y=0,z=0)
>>> move_to_ang(MODE_TYPE.MOVJ,x=0)
>>> jnts = Joints(j1=50,j2=10,j3=0,j4=0)
>>> move_to_ang(MODE_TYPE.MOVL,position=pos)

```

move_to_xyz(mode: [MOVE_TYPE](#), x=None, y=None, z=None, r=None, wait=True, position: [Position](#) = None) [\[source\]](#)

Move robotic arm to the (x,y,z) position in the Cartesian plane and set the end effector angle to r.

Parameters:

mode : *bot.MOVE_TYPE*

Type of movement to be performed. Valid options { MOVE_TYPE.JUMP, MOVE_TYPE.MOVJ, MOVE_TYPE.MOVL }.

x : *float, optional*

Target end effector x-coordinate. Defaults to the current x-coordinate if None.

y : *float, optional*

Target end effector y-coordinate. Defaults to the current y-coordinate if None.

z : *float, optional*

Target end effector z-coordinate. Defaults to the current z-coordinate if None.

r : *float, optional*

Target end effector rotation servo orientation. Defaults to 0.

wait : *bool, optional*

If True, wait for the previous movement to complete before executing this new one. Defaults to True.

position : *bot.Position, optional*

A Position object containing x, y, z, and r values. If provided, overrides x, y, z, and r parameters.

Returns:

Robot ptp movement instruction.

Raises:**ValueError**

If both position and (x, y, z) args are None, or if an invalid movement type is provided.

Examples

```
>>> move_to_xyz(MODE_TYPE.MOVJ,x=200,y=200,z=10)
>>> move_to_xyz(MODE_TYPE.MOVJ,z=0)
>>> pos = Position(x=0,y=200,z=0,r=0)
>>> move_to_xyz(MODE_TYPE.JUMP,position=pos)
```

set_home(x, y, z, r=0.0)[\[source\]](#)

Defines the dobot robotic arm home position in the cartesian plane.

Parameters:

x : *float*

y : *float*

z : *float*

r : *float, default = 0.*

↪ See also

`bot.home`

suck(enable=False)[\[source\]](#)

Control suction in the end effector.

Parameters:

enable: *bool, default = False*

Notes

Will only have effect if the suction module is mounted and the pressure module is connected.

```
exception magician.bot.BotBoundsException(message='Bot Bounds Exception')
```

Bases: `Exception`

[\[source\]](#)

Exception raised when bot movement is set to go out of bounds.

Attributes:

message : *str, optional*

A description of the error. The default is "Bot Bounds Exception".

Examples

```
>>> raise BotBoundsException("j1 angle out of bonds, must be in range [-125°,125°]")
>>> raise BotBoundsException()
```

```
exception magician.bot.BotException(message='Bot Exception')
```

[\[source\]](#)

Bases: `Exception`

Exception raised for errors related to the bot operations.

Attributes:

message : *str, optional*

A description of the error. The default is "Bot Exception".

Examples

```
>>> raise BotException("Device not found.")
>>> raise BotException()
```

```
class magician.bot.Joints(j1: float, j2: float, j3: float, j4:
float)
```

[\[source\]](#)

Bases: `NamedTuple`

Defines a position in the Joints Plane.

Parameters:

j1 : *float*

j1 joint angle in the Joints Plane.

j2 : *float*

j2 joint angle in the Joints Plane.

j3 : *float*

j3 joint angle in the Joints Plane.

j4 : *float*

j4 joint (effector) angle in the Joints Plane.

Methods

<code>count</code> (value, /)	Return number of occurrences of value.
<code>in_radians</code> ()	Returns joints angles in radians.
<code>index</code> (value[, start, stop])	Return first index of value.

`in_radians()` → [Joints](#)

[\[source\]](#)

Returns joints angles in radians.

j1: *float*

Alias for field number 0

j2: *float*

Alias for field number 1

j3: *float*

Alias for field number 2

j4: *float*

Alias for field number 3

```
class magician.bot.MODE_PTP(value, names=<not given>, *values, module=None,
qualname=None, type=None, start=1, boundary=None)
```

[\[source\]](#)

Bases: `IntEnum`

Enumeration for types of Point to Point Movement message payload for the Dobot Magician. The MODE_PTP enum defines the Point to Point number code definitions.

Parameters:

JUMP_XYZ : int

JUMP mode, (x, y, z, r) is the target point in Cartesian coordinate system.

MOVJ_XYZ : int

MOVJ mode, (x, y, z, r) is the target point in Cartesian coordinate system.

MOVL_XYZ : int

MOVL mode, (x, y, z, r) is the target point in Cartesian coordinate system.

JUMP_ANGLE : int

JUMP mode, (x, y, z, r) is the target point in Joint coordinate system.

MOVJ_ANGLE : int

MOVJ mode, (x, y, z, r) is the target point in Joint coordinate system.

MOVL_ANGLE : int

MOVL mode, (x, y, z, r) is the target point in Joint coordinate system.

MOVJ_INC : int

MOVJ mode, (x, y, z, r) is the angle increment in Joint coordinate system.

MOVL_INC : int

MOVL mode, (x, y, z, r) is the Cartesian coordinate increment in Joint coordinate system.

MOVJ_XYZ_INC : int

MOVJ mode, (x, y, z, r) is the Cartesian coordinate increment in Cartesian coordinate system.

JUMP_MOVL_XYZ:

JUMP mode, (x, y, z, r) is the Cartesian coordinate increment in Cartesian coordinate system.

Attributes:

`denominator`

the denominator of a rational number in lowest terms

`imag`

the imaginary part of a complex number

`numerator`

the numerator of a rational number in lowest terms

`real`

the real part of a complex number

Methods

<code>as_integer_ratio</code> (/)	Return a pair of integers, whose ratio is equal to the original int.
<code>bit_count</code> (/)	Number of ones in the binary representation of the absolute value of self.
<code>bit_length</code> (/)	Number of bits necessary to represent self in binary.
<code>conjugate</code>	Returns self, the complex conjugate of any int.
<code>from_bytes</code> (/, bytes[, byteorder, signed])	Return the integer represented by the given array of bytes.
<code>is_integer</code> (/)	Returns True.
<code>to_bytes</code> (/[, length, byteorder, signed])	Return an array of bytes representing an integer.

Notes

Do the chosen names make sense? No ... But this is what's given by the manufacturer in the API description. That's also why this enum is not meant to be used by people using this lib, only for those maintaining it.

```
class magician.bot.MOVE_TYPE(value, names=<not given>, *values, module=None,
qualname=None, type=None, start=1, boundary=None) \[source\]
```

Bases: `IntEnum`

The `MOVE_TYPE` enum defines the types of movement available for the Robotic Arm.

Parameters:

MOVJ : *int*

Joint movement. From point A to point B, each joint will run from initial angle to its target angle, regardless of the trajectory.

MOVL : *int*

Rectilinear movement. The joints will perform a straight line trajectory from point A to point B.

JUMP : *int*

From point A to point B the end effector will lift upwards by amount of Height (in mm) and move horizontally to a point that is above B by Height and then move down to Point B.

Attributes:

`denominator`

the denominator of a rational number in lowest terms

`imag`

the imaginary part of a complex number

`numerator`

the numerator of a rational number in lowest terms

`real`

the real part of a complex number

Methods

<code>as_integer_ratio</code> (/)	Return a pair of integers, whose ratio is equal to the original int.
<code>bit_count</code> (/)	Number of ones in the binary representation of the absolute value of self.
<code>bit_length</code> (/)	Number of bits necessary to represent self in binary.
<code>conjugate</code>	Returns self, the complex conjugate of any int.

<code>from_bytes</code> (/, bytes[, byteorder, signed])	Return the integer represented by the given array of bytes.
<code>is_integer</code> (/)	Returns True.
<code>to_bytes</code> (/[, length, byteorder, signed])	Return an array of bytes representing an integer.

Examples

```
>>> linear_movement = MOVE_TYPE.MOVL
>>> move_to_xyz(linear_movement,x=200,y=0,z=0)
```

`class` `magician.bot.Pose`(`position`: [Position](#), `joints`: [Joints](#))

[\[source\]](#)

Bases: `NamedTuple`

NamedTuple that defines position based on the Position and Joints Objects.

Parameters:

position : *Position*

joints : *Joints*

Methods

<code>count</code> (value, /)	Return number of occurrences of value.
<code>index</code> (value[, start, stop])	Return first index of value.

 See also

`bot.Position`

`bot.Joints`

joints: [Joints](#)

Alias for field number 1

position: Position

Alias for field number 0

```
class magician.bot.Position(x: float, y: float, z: float, r: float)
```

[\[source\]](#)

Bases: `NamedTuple`

Defines a position in the Cartesian Plane.

Parameters:

- x : float**
x-coordinate in the Cartesian Plane.
- y : float**
y-coordinate in the Cartesian Plane.
- z : float**
z-coordinate in the Cartesian Plane.
- r : float**
Effector orientation angle.

Methods

<code>count</code> (value, /)	Return number of occurrences of value.
<code>index</code> (value[, start, stop])	Return first index of value.

r: float
Alias for field number 3

x: float
Alias for field number 0

y: float
Alias for field number 1

z: float
Alias for field number 2

magician.message module

`class magician.message.Message(b=None)` [\[source\]](#)

Bases: `object`

Methods

<code>bytes</code>	
<code>refresh</code>	

`bytes()` [\[source\]](#)

`refresh()` [\[source\]](#)